

EXPERIMENT NO: 2 Analysis of Quick Sort on Nearly Sorted Datasets

Aim: To evaluate the behavior of Quick Sort on nearly sorted datasets and compare its performance with random datasets.

Objective:

- Implement Quick Sort algorithm.
- Measure execution time on nearly sorted and random datasets.
- Analyze the impact of pivot selection on performance.
- Suggest techniques to improve efficiency.

Problem Statement:

Evaluate the behavior of Quick Sort on nearly sorted datasets by implementing the algorithm and measuring execution time. Compare the results with random input datasets. Analyze how pivot selection impacts performance degradation. Interpret findings and justify techniques to improve efficiency.

Algorithm:

1. Choose the first element as pivot.
2. Partition elements into left and right subarrays.
3. Recursively apply Quick Sort on partitions.
4. Measure execution time for nearly sorted and random datasets.

Observation:

Quick Sort executed slower on nearly sorted datasets because choosing the first element as pivot creates highly unbalanced partitions, leading to $O(n^2)$ complexity. Random datasets generally provide more balanced partitions and better performance.

Execution Time Comparison:

Nearly Sorted Dataset : _____ seconds

Random Dataset : _____ seconds

Time Complexity:

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n^2)$

Techniques to Improve Efficiency:

- Randomized Pivot Selection
- Median-of-Three Pivot Selection
- Hybrid Sorting Approaches

Result:

The Quick Sort algorithm was successfully implemented and analyzed. The performance degraded for nearly sorted datasets due to poor pivot selection. Improved pivot selection techniques can significantly enhance efficiency.

Conclusion:

Quick Sort performance heavily depends on pivot selection. Using advanced pivot strategies helps maintain $O(n \log n)$ performance and avoids worst-case behavior.